コーディング特化LLMを用いたソフトウェアリバースエンジニアリングの性能評価

川越 謙宏[1,a)]　小林 良太郎[1]

概要：マルウェア解析では Large Language Model (LLM) エージェントが Model Context Protocol (MCP) を介してツールを自動操作して解析する手法が一般的になっている．LLM は高性能なモデルを用いれば複雑なタスクもこなせるが，コストを要する．本研究ではトークン当たりのコストが少ない SpaceX 社の傘下である Anysphere 社のコーディング特化モデルである Composer 2.5 を用いて，ソフトウェアリバースエンジニアリングの性能評価を行った．比較対象として，一般的に用いられる OpenAI 社や Anthropic 社のモデルを使用し，難読化されたバイナリやマルウェアを解析させた．また，解析手順やツールの利用方法を記述した Agent Skills を付与した場合と付与しない場合について，解析性能の変化を評価した．

キーワード：静的解析，LLM エージェント，MCP

Performance Evaluation of Software Reverse Engineering Using a Coding Focused Large Language Model

Akihiro Kawagoe[1,a)]　Ryotaro Kobayashi[1]

Abstract: In malware analysis, approaches in which Large Language Model (LLM) agents automatically operate tools through the Model Context Protocol (MCP) have become increasingly common. Although high-performance LLMs can handle complex tasks, their use incurs substantial costs. In this study, we evaluated the performance of Composer 2.5, a coding-specialized model developed by Anysphere, a subsidiary of SpaceX, for software reverse engineering. For comparison, models from widely used providers such as OpenAI and Anthropic were employed to analyze obfuscated binaries and malware. Furthermore, we investigated how the presence or absence of Agent Skills describing analysis procedures and tool usage affected analysis performance.

Keywords: Static Analysis, LLM Agent, MCP

1. はじめに

Large Language Model (LLM) の発展により，Ghidra や IDA Pro などの Software Reverse Engineering (SRE) ツールを Model Context Protocol (MCP) を介して LLM エージェントが自動操作し，解析することが可能になっている．

OpenAI 社の GPT-5.6 Terra は 100 万トークンあたり，入力に$2，出力に$12 必要である．一方で，高度な推論が可能な GPT-5.6 Sol は入力に$5，出力に$30 必要である [1]．また Anthropic 社の Claude Sonnet 5 は 100 万トークンあたり，入力に$2，出力に$10 必要である．一方で，高度な推論が可能な Claude Fable 5 は入力に$10，出力に$50 必要である [2]．上述のように高性能なモデルの利用には比較的高い金銭的コストを要する．高性能なモデルを SRE に用いると精度の高い解析が可能であるが，資源は有限である．そのため精度を保ちつつコストを削減することが重要である．

Cursor は SpaceX 社の傘下である Anysphere 社が開発

[1] 工学院大学
Kogakuin University, Shinjuku, Tokyo 163-8677, Japan
[a)] em25009@ns.kogakuin.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

した AI を組み込んだ IDE である．Cursor では Anysphere 社独自のモデルである Composer が利用可能である．最新モデルである Composer 2.5 はコーディングタスク用に開発されたモデルであり，Moonshot AI 社が開発したオープンモデルである Kimi K2.5 を基盤とし，独自に強化学習を行っている [3]．Composer 2.5 は 100 万トークンあたり，入力に$0.5，出力に$2.5 必要である [4]．また優先的に処理を行う Fast Mode では 6 倍の費用を要する．

Ghidra や IDA Pro を用いた SRE は Disassemble 結果から元のソースコードを予測した多くの Decompile 出力を基にソフトウェアの挙動を解析する．この性質からコーディング特化モデルである Composer 2.5 はできる限り精度を落とさず少ないコストで SRE タスクの実行が可能であるという仮説を立てた．本研究では LLM エージェントとして Composer 2.5 を利用し MCP を介して Ghidra を操作させ，SRE 能力の評価を行った．

本論文の構成は，まず，第 2 章で LLM と SRE に関する研究を述べ，その後第 3 章で本研究の調査方法を述べ，その後第 4 章で検証結果を示し，最後に第 5 章で考察を述べ，第 6 章で結論を述べる．

2. 関連研究

2.1 バイナリ解析 AI エージェント用 LLM ベンチマークの試作

大坪らは LLM エージェントによるバイナリ解析はプロンプトの不備や外部ツールの精度など，様々な要因が結果に影響しているため，解析精度のボトルネックとなる工程の特定が困難であるという前提のもと，LLM エージェントの最上流の能力であるバイナリシーケンスの正確な解釈と操作を定量的に評価するための動的ベンチマーク手法を提案した [5]．考察として，バイナリデータの提示形式というコンテキストエンジニアリングがモデルの能力を左右する要因となることが示された．また 2025 年後半以降の新しいモデルはバイナリシーケンスの理解能力が劇的に向上していることなどが示された．

2.2 DecBench

Basque が率いるジョージア大学の Noelo Lab は Control Flow Graph (CFG)，型，再コンパイル性などの観点から Decompiler の正確性を評価するための実験的なベンチマークを作成した．DecBench では既存の Decompiler に加えて LLM のスコアリングを行った [6]．データセットには LLM 評価用の小規模な Sample-Set が含まれており，Sample-Set では CFG は GPT-5.6 Sol が，型では Claude Opus 4.8 が最も精度が高かった．また生成された Decompile 出力に対して再コンパイルが可能になるように共通の処理を行った結果，LLM ベースでない Decompiler のスコアが高く，LLM は 0%であった．

なお DecBench で使用されたデータセットはスコアの計測が可能であったものを分母としているため，Decompiler や LLM によって分母が異なっている．また CFG や型は正答にたどり着くまでの距離をスコアに使用している．

3. 調査方法

本章では調査手法について述べる．

3.1 検証検体

2 種類の検体で検証を行った．1 つはコマンドライン引数で与えた文字列をカスタム AES で暗号化し，正しい入力が与えられた場合に Correct!と出力する C で作成したプログラムである．Nishizaka らの研究 [7] から，標準仕様と僅かに異なるアルゴリズムを用いることで LLM の解析を妨害し誤答へと誘導することを意図した．

```
S-Box_obf ← 0x42 で XOR された S-Box
Swap_Seed ← Swap 箇所算出用の Seed 値配列
S-Box ← S-Box_obf ⊕ 0x42
for i ← 0 to 6 step 2 do
    p ← (5i) mod 16
    q ← (5i + 1) mod 16
    x ← Swap_Seed[i] ⊕ K[p]
    y ← Swap_Seed[i + 1] ⊕ K[q]
    Swap(S-Box[x], S-Box[y])
```

図 1 S-Box の Swap 処理
Fig. 1 S-Box Swap Processing

図 1 が通常の AES からの変更箇所である．S-Box を 0x42 で XOR した値をバイナリ内にハードコードし，一度復号処理を行う．その後，復号後の S-Box に対して Swap 処理を行って暗号化を行っている．また入力された文字列は，ランダムな順番で各文字の正誤判定を行うようになっている．このプログラムを Collberg らが開発した難読化ツール Tigress[8] を用いて難読化処理を行ったものを検証に使用した．この検体は難読化された複雑な検体を解析し，挙動の理解能力を計測すること，理解した挙動を応用して正しい入力を求めるためのスクリプトを生成する能力を計測することを意図している．難読化では Control Flow Flattening (CFF), Mixed Boolean Arithmetic (MBA), Virtualize, Opaque Predicates を使用した．この検体を以降，検証検体 1 と呼ぶ．

2 つ目の検体ではマルウェアを解析させ，レポートを生成させた．マルウェアには Advanced Persistent Threat (APT) 41 の DodgeBox を使用した．この検体を以降，検証検体 2 と呼ぶ．

© 2026 Information Processing Society of Japan

表 1 検証検体 2 のハッシュ値
Table 1 Hash Values of Verification Sample 2

ファイル名	MD5 ハッシュ値
taskhost.exe	b3067f382d70705d4c8f6977a7d7bee4
sbiedll.dll	d72f202c1d684c9a19f075290a60920f
sbiedll.dat	294cc02db5a122e3a1bc4f07997956da

表 1 に検証検体 2 のハッシュ値を示す．正規の実行ファイルである taskhost.exe が Dynamic Link Library (DLL) サイドローディングで悪意のある sbiedll.dll を読み込む．sbiedll.dll では AES-CFB で sbiedll.dat の復号を行う．復号されたバックドアである DLL 形式のファイルをリフレクティブ DLL インジェクションで読み込むという挙動であることが Zscaler 社が公開している情報から明らかである [9]．ただ, sbiedll.dat の復元時に使用する IV はレジストリ HKLM\SOFTWARE\Microsoft\Cryptography にセットされているマシンごとに異なる GUID の MD5 ハッシュ値を使用している．そのため，解析結果から得た情報では sbiedll.dat の復号はできないが，復号したものが Portable Executable (PE) であることから先頭 16 バイトの平文を推測することで IV を復元し，sbiedll.dat の復号が可能である．また解析からレポート作成までの Agent Skills を与えた場合の振る舞いの変化についても検証した．

3.2 使用した Skill

検証検体 2 では LLM の振る舞いの変化を計測するために，マルウェア解析を行い実行フロー，難読化，暗号化，C2，IOCs など章立てて HTML レポートを作成する Skill，シェルコードなどペイロードが埋め込まれていた場合，抽出を行い継続的に解析を行う Skill，疑似 C で制御フローを作成する Skill，複数検体を解析する際，総括となるレポートを軸に検体ごとのレポートを作成する Skill，CFF などの難読化がされていた場合，Angr や Unicorn などで解決を行う Skill を作成し，使用した．

3.3 検証環境

表 2 検証環境
Table 2 Verification Environment

項目	バージョン
OS	Windows Hyper-V 上の Windows 10 Pro
Ghidra	12.1.2
Ghidra MCP	6.0.0

表 2 は検証に使用した環境である．Ghidra MCP は GitHub 上に複数存在するが，今回は bethington らが開発した Ghidra MCP[10] を使用した．また Ghidra は静的解析だけでなく Debugger を用いた動的解析も可能であり Ghidra MCP にも含まれている．今回，検証検体としてマルウェアを使用したため, 動的解析関連のツールの使用を禁止し，Ghidra Script と Python の実行を許可し，Symbolic Execution と Emulation が可能な環境で検証した．また検証は IDE である Cursor 上で行っている．GPT や Claude の検証も Cursor 上で行っているため，ハーネスの違いが結果に影響することがある．

3.4 モデル

表 3 使用したモデル
Table 3 Models Used

モデル名	開発元
GPT-5.6 Sol	OpenAI
Claude Opus 5	Anthropic
Claude Fable 5	Anthropic
Composer 2.5	Anysphere
Cursor Grok 4.6	SpaceX, Anysphere
Kimi K3	Moonshot AI

実験には表 3 のモデルを使用した．GPT のように Effort を選択可能なモデルは，小規模な SRE タスクでは Medium を，マルウェア解析を行いレポートを生成させる際は Extra High を選択した．例外的に Kimi K3 は Effort が Low，High，Max しか選べないため，すべてのタスクで High を使用した．また優先的に処理を行う Fast Mode が利用可能なモデルは Fast Mode で，Thinking Mode が利用可能な場合は Thinking Mode を有効にして計測を行った．

3.5 検証 1

検証 1 は検証検体 1 で行った検証である．検証 1 の詳細を以下に示す．

3.5.1 評価項目

表 4 検証 1 の評価項目
Table 4 Evaluation Items in Verification 1

項目	意図
AES ロジックの発見	挙動の理解能力
S-Box の Swap の発見	LLM を誤解させる罠への耐性
正しい入力の導出	応用能力

表 4 が検証検体 1 の評価項目である．AES ロジックの発見は Decompile 出力から挙動を大まかに理解できているかを評価することを意図している．S-Box の Swap の発見は，LLM が大まかな挙動理解で判断をし，意図的な誤推論への誘導耐性を評価することを意図している．正しい入力の導出は，理解した挙動から動的解析が不可能な状況下での応用能力を評価することを意図している．後述の条件で検証を行い，生成されたレポートを上から順に採点を行い，満たしていた場合に 1 点を加算し最大 3 点で評価を行った．

© 2026 Information Processing Society of Japan

3.5.2 条件

試行回数は最大 3 回とした．各試行では最大 10 分間作業を行い，10 分が経過した段階でレポートの生成を依頼した．各社の最上位モデルで検証を行った．3 回の試行で生成されたレポートを評価し，最も優れたものを最終的な点数とした．例外的に Claude は最上位モデルである Claude Fable 5 に加えて，Claude Opus 5 で検証を行った．またエージェントが Python を実行する場合に正しいパスを与えるための環境設定用 Skill を与えていた．

3.6 検証 2

検証 2 は検証検体 2 で行った検証である．検証 2 の詳細を以下に示す．

3.6.1 評価項目

表 5 検証 2 の評価項目
Table 5 Evaluation Items in Verification 2

項目	採点対象の記述
Config	使用されている暗号アルゴリズム Config の復号
API Hashing	使用されているハッシュアルゴリズム 呼び出されている API の言及
sbiedll.dat の復号	使用されている暗号アルゴリズム 復号の成否
DLL の抽出	2 種類の DLL を抽出
耐解析，検知回避	DLL サイドローディング Reflective DLL Loading 実行時の引数チェック Control Flow Guard の無効化

表 5 が検証 2 における採点項目と採点対象の記述である．採点対象の記述がレポートに記載されていた場合，1 点を加算する．Config から DLL の抽出までが，どこまで自律的に調査を行えたかの段階を表しており，モデルの複雑なタスクへの評価指標となる．耐解析，検知回避はレポートとして必要な情報を適切に抽出できているかを判断するための指標である．

3.6.2 条件

Claude Fable 5, GPT-5.6 Sol, Composer 2.5, Cursor Grok 4.6, Kimi K3 を使用し，1 回の指示でレポートを生成させた．Effort が存在するものは Max から 1 段階下げたものを，Composer 2.5 と Cursor Grok 4.6 は Fast Mode を使用した．まずエージェントが Python を実行する場合に正しいパスを与えるための環境設定用 Skill のみを与え，レポートを生成させた．その後，環境をリセットし，レポートのテンプレートを含む Skill を与え，レポートを作成させた．また試行回数は 1 回である．

4. 調査結果

検証 1 と検証 2 の結果を以下に示す．

4.1 検証 1

表 6 検証 1 における各モデルの評価可否
Table 6 Evaluability of Each Model in Verification 1

モデル	評価可否
Claude Fable 5	可
Claude Opus 5	可
GPT-5.6 Sol	不可
Composer 2.5	可
Cursor Grok 4.6	不可
Kimi K3	可

表 6 が各モデルの評価可否である．GPT-5.6 Sol および Cursor Grok 4.6 Fast はセキュリティリスクを検出され，作業が中断されたため評価不能であった．

表 7 検証 1 における消費トークン，費用，点数
Table 7 Token Usage, Cost, and Score in Verification 1

モデル	消費トークン		費用	スコア
Claude Fable 5	C-Read	1,491,747	$4.87	3
	C-Write	104,843		
	Input	48		
	Output	41,338		
Claude Opus 5	C-Read	3,851,780	$4.33	3
	C-Write	220,743		
	Input	102		
	Output	41,053		
Composer 2.5 Fast	C-Read	5,008,128	$2.53	2
	C-Write	0		
	Input	327,037		
	Output	86,332		
Kimi K3	C-Read	1,909,596	$1.56	3
	C-Write	0		
	Input	87,790		
	Output	48,485		

表 7 が消費したトークンと費用，採点結果である．C-Read は Cache Read，C-Write は Cache Write を意味する．C-Write が 0 のものは計測ミスではなく，元々 C-Write を使用しないモデルである．Kimi K3 は 3 回目の試行で正しい入力にたどり着いた．Composer 2.5 は Fast Mode であり，すべての試行で Swap ロジックに気付いたが，制限時間内に正しい入力にたどり着かなかった．また Claude Fable 5 と Claude Opus 5 はどちらも 1 度の試行で正しい入力にたどり着いた．

© 2026 Information Processing Society of Japan

今回 Windows 環境で解析を行わせているため，Linux の実行環境がない．そのため，すべてのモデルにおいて Unicorn によるエミュレータを Python で作成し正しい入力の検証を行う傾向が見られ，その成否が正しい入力の導出に影響した．

4.2 検証 2

表 8 検証 2 における各モデルの評価可否
Table 8 Evaluability of Each Model in Verification 2

モデル	評価可否
Claude Fable 5	不可
GPT-5.6 Sol	可
Composer 2.5	可
Cursor Grok 4.6	可
Kimi K3	可

表 8 が各モデルの評価可否である．Claude Fable 5 はセキュリティリスクを検出され，作業が中断されたため，評価不能であった．表 9 は検証 2 における Skill を使用しなかった場合の消費トークンと費用である．GPT-5.6 Sol は Thinking Mode が存在するため，非常に多くのトークンを消費していることが表からわかる．Effort のある Kimi K3 が次点でトークンの消費が多いが，Effort のある Cursor Grok 4.6 は Composer 2.5 よりも僅かに消費トークンが多く，最も消費トークンが少なかったモデルは Composer 2.5 である．このことから複雑なタスクに対して単純な指示を与えた場合，モデルの Effort と Thinking Mode の有無が LLM エージェントの作業量に大きく影響している．

表 9 検証 2 における Skill を使用しなかった場合の消費トークンと費用
Table 9 Token Usage and Cost without Skill in Verification 2

モデル	消費トークン		費用
GPT-5.6 Sol	C-Read	26,695,707	$20.73
	C-Write	474,369	
	Input	284,333	
	Output	99,780	
Composer 2.5 Fast	C-Read	1,192,768	$0.67
	C-Write	0	
	Input	89,458	
	Output	22,920	
Cursor Grok 4.6 Fast	C-Read	1,207,168	$2.19
	C-Write	0	
	Input	118,129	
	Output	42,542	
Kimi K3	C-Read	8,165,276	$5.81
	C-Write	0	
	Input	514,767	
	Output	120,829	

表 10 検証 2 における Skill を使用した場合の消費トークンと費用
Table 10 Token Usage and Cost with Skill in Verification 2

モデル	消費トークン		費用
GPT-5.6 Sol	C-Read	34,423,117	$35.11
	C-Write	1,081,495	
	Input	1,088,976	
	Output	166,804	
Composer 2.5 Fast	C-Read	6,061,672	$4.01
	C-Write	0	
	Input	663,124	
	Output	114,714	
Cursor Grok 4.6 Fast	C-Read	7,510,016	$9.64
	C-Write	0	
	Input	259,214	
	Output	91,320	
Kimi K3	C-Read	12,557,280	$7.36
	C-Write	0	
	Input	549,085	
	Output	129,867	

表 10 は，検証 2 における Skill を使用した場合の消費トークンと費用である．Skill を与えた場合，Effort や Thinking Mode のない Composer 2.5 はトークン消費が約 500 万増加していた．また Cursor Grok 4.6 もトークンの消費量が大きく増加した．

表 11 と表 12 が Skill の有無ごとの評価結果である．Skill 無しの場合，最もトークン消費が多かった GPT-5.6 Sol と Kimi K3 が最も高得点だった．この 2 つのモデルは sbiedll.dat の復号に成功している．GPT 5.6 は復号後のバイナリ解析を継続的に行っているが，Kimi K3 は復号で作業を中断していた．しかし GPT-5.6 Sol は実行時の引数チェックを記述しなかったが Kimi K3 は記述したため同じ点数となった．Skill を与えた場合，Kimi K3 は Skill 無しと同様の振る舞いをし，同じ点数となった．GPT-5.6 Sol は DLL サイドローディングの記述がなくなり 1 点下がった．Composer 2.5 は Skill を与えたことで Config の復号に成功した．レポートの文章量が増加し，スコアも上昇したものの，Skill の有無に関わらず，AES-CFB や FNV-1a などのアルゴリズムの検出が苦手である傾向が見られた．また点数には直接的な影響がないが Skill に含まれている例がノイズとなり Composer 2.5 が誤った推論をしていた．他のモデルではその傾向が見られなかった．また耐解析，検知回避では Control Flow Guard の無効化がすべてのモデルで Skill の有無に関わらず，レポートに記述が一切なかった．

© 2026 Information Processing Society of Japan

表 11 Skill を使用しなかった場合の検証 2 の結果
Table 11 Results of Verification 2 without Skill

モデル	Config	API Hashing	sbiedll.dat の復号	DLL の抽出	耐解析，検知回避
GPT-5.6 Sol	2	2	2	1	2
Composer 2.5	0	0	0	0	2
Cursor Grok 4.6	0	2	1	0	2
Kimi K3	2	2	2	0	3

表 12 Skill を使用した場合の検証 2 の結果
Table 12 Results of Verification 2 with Skill

モデル	Config	API Hashing	sbiedll.dat の復号	DLL の抽出	耐解析，検知回避
GPT-5.6 Sol	2	2	2	1	1
Composer 2.5	2	0	1	0	3
Cursor Grok 4.6	2	2	1	0	3
Kimi K3	2	2	1	0	3

5. 考察

検証 1 から，Composer 2.5 は挙動理解には優れているが，応用して正しい入力を導出するには至らなかった．また検証 2 から，複雑なタスクを自律的に行う能力やアルゴリズムを推論する能力は他のモデルより低く，自動解析エージェントとしての能力は十分でないと考えられる．Skill を与えることで点数は上昇したが，Skill に例として記述されていたノイズによって誤った推論をした．

表 13 Skill 修正後の Composer 2.5 のスコア
Table 13 Scores of Composer 2.5 After Skill Revision

Config	API Hashing	sbiedll.dat	DLL	耐解析，検知回避
0	0	0	0	1

そのため Skill を修正し Composer 2.5 で再評価を行った．表 13 がその結果である．ノイズによる誤誘導はなくなったが，点数が大きく低下した．LLM の性質上，試行回数によって点数が増加する可能性もあるが，複雑なタスクにおいて，Skill によって出力トークンが増えても精度が向上することはなかった．従って，Composer 2.5 では抽象的でない的確な指示を与え，コンテキストが大きくなり忘却が起きないようタスクを絞って指示を与えることが重要である．

また本研究の課題として LLM の性質上，実験の試行回数が不足していることが挙げられる．また評価についても性能を正しく評価するための手法を考え，より客観的に評価する必要がある．

6. 結論

本研究では Composer 2.5 の SRE 性能を簡易的に評価した．検証結果から小規模なバイナリの挙動理解が可能であるものの，理解した情報の応用や，複雑なタスクの実行が困難であることが判明した．LLM の性能を正しく評価するために検証検体と検証回数を増やすことが今後の課題である．

謝辞 本研究の一部は，JSPS 科研費 26K14827 の支援により行った．

参考文献

[1] OpenAI, “Models,” https://developers.openai.com/api/docs/models (Accessed 2026-08-11).
[2] Anthropic, “モデル概要,” https://platform.claude.com/docs/ja/about-claude/models/overview (Accessed 2026-08-11).
[3] Anysphere, “Composer,” https://cursor.com/ja/blog/composer-2-5 (Accessed 2026-08-11).
[4] Anysphere, “モデルと料金,” https://cursor.com/ja/docs/models-and-pricing (Accessed 2026-08-11).
[5] 大坪雄平, 大塚玲, “バイナリ解析 AI エージェント用 LLM ベンチマークの試作,” 人工知能学会第二種研究会資料, Vol. 2026, No. SEC-005, 2026.
[6] Z. L. Basque, “DecBench,” https://decbench.com (Accessed 2026-08-20).
[7] R. Nishizaka, Y. Fujiwara, T. Shimizu, K. Kato and Y. Sugiyama, “Towards LLM-Resistant Software Protection: Agent Failure Patterns in CTF Reverse Engineering,” Proceedings of the Workshop on Binary Analysis Research (BAR) 2026.
[8] C. Collberg, “the tigress c obfuscator,” https://tigress.wtf (Accessed 2026-08-18).
[9] Zscaler, “DodgeBox: A deep dive into the updated arsenal of APT41 |Part 1,” https://www.zscaler.com/blogs/security-research/dodgebox-deep-dive-updated-arsenal-apt41-part-1 (Accessed 2026-08-18).
[10] bethington, “ghidra-mcp,” https://github.com/bethington/ghidra-mcp (Accessed 2026-08-11).

© 2026 Information Processing Society of Japan